Solution: Zigzag Iterator

Problem Statement

Given two `1d` vectors, implement an iterator to return their elements alternately.

For example, given two `1d` vectors:

```
v1 = [1, 2]
v2 = [3, 4, 5, 6]
```

By calling `next()` repeatedly until `hasNext()` returns `false`, the order of elements returned by next should be: [1, 3, 2, 4, 5, 6].

Example 1:

```
i = ZigzagIterator([1, 2], [3, 4, 5, 6])
print(i.next())  # returns 1
print(i.next())  # returns 3
print(i.next())  # returns 2
print(i.next())  # returns 4
print(i.next())  # returns 5
print(i.next())  # returns 6
print(i.hasNext())  # returns False
```

Example 2:

```
i = ZigzagIterator([1, 2, 3, 4], [5, 6])
print(i.next())  # returns 1
print(i.next())  # returns 5
print(i.next())  # returns 2
print(i.next())  # returns 6
print(i.next())  # returns 3
print(i.next())  # returns 4
print(i.hasNext())  # returns False
```

Example 3:

```
i = ZigzagIterator([1, 2], [])
print(i.next())  # returns 1
print(i.next())  # returns 2
print(i.hasNext())  # returns False
```

Solution

The Zigzag Iterator can be implemented using a Queue of Iterators. Here is a simple idea of how we can implement the Zigzag Iterator:

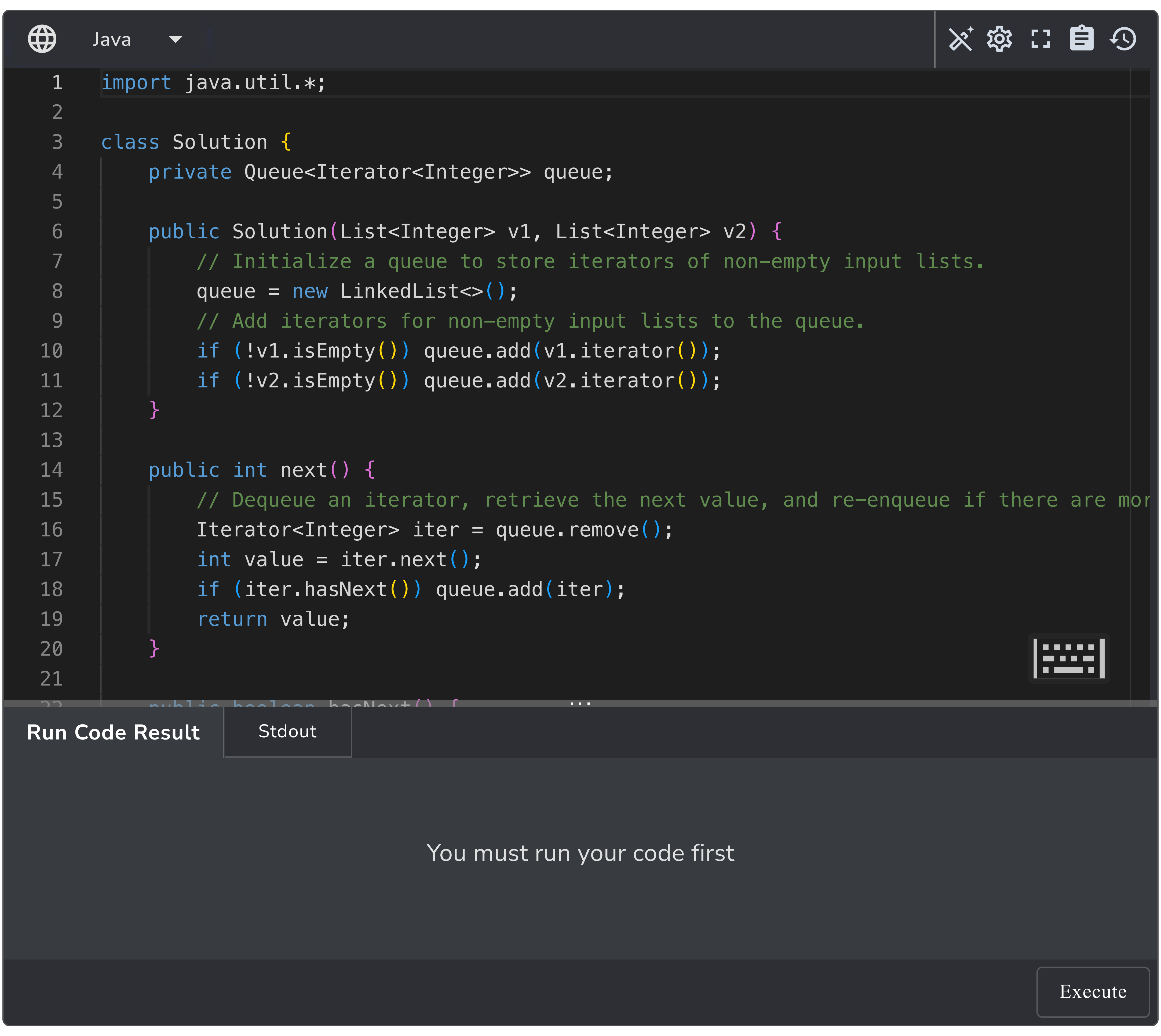
1. Push the iterators of v1 and v2 into the Queue in order.
2. For each next() operation, dequeue an iterator from the Queue, retrieve the current element from that iterator, then re-queue that iterator if there are still elements left in the iterator.

Detailed Walkthrough:

1. Initialize the queue and insert the iterators of v1 and v2 only if they have elements.
2. In the `next()` function, pop an iterator from the front of the queue. Get the next element from the iterator, then re-insert the iterator back into the queue if there are still elements left in the iterator.
3. In the `hasNext()` function, simply check if the queue is empty.

Code

Here is how we can implement this algorithm:



```java
import java.util.*;

class Solution {
    private Queue<Iterator<Integer>> queue;

    public Solution(List<Integer> v1, List<Integer> v2) {
        // Initialize a queue to store iterators of non-empty input lists.
        queue = new LinkedList<>();
        // Add iterators for non-empty input lists to the queue.
        if (!v1.isEmpty()) queue.add(v1.iterator());
        if (!v2.isEmpty()) queue.add(v2.iterator());
    }

    public int next() {
        // Dequeue an iterator, retrieve the next value, and re-enqueue if there are mor
        Iterator<Integer> iter = queue.remove();
        int value = iter.next();
        if (iter.hasNext()) queue.add(iter);
        return value;
    }

```

Time and Space Complexity:

The `next()` and `hasNext()` operations have a time complexity of O(1).

The space complexity is O(m), where m is the total number of vectors - since each vector is represented as a separate iterator in the deque.

← Previous
Problem 5: Zigzag Iterator

Next →
Problem 6: Max Of All Subarrays Of Size 'K'

☐ Mark as Completed